TASK 1: IMPORT LIBRARIES AND LOAD DATASET

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import warnings

from statsmodels.tsa.stattools import adfuller
from statsmodels.tsa.arima.model import ARIMA
from sklearn.metrics import mean_squared_error, mean_absolute_error

warnings.filterwarnings("ignore")

# Download and read Airline Passengers dataset
url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/airline-passengers.csv"

df = pd.read_csv(
    url,
    parse_dates=["Month"],
    index_col="Month"
)

df.index = pd.DatetimeIndex(df.index, freq="MS")

print("Dataset loaded successfully!")
display(df.head())
```

```
Dataset loaded successfully!
```

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

TASK 2: EXPLORE THE DATASET

```
print("\nFirst five records:")
display(df.head())

print("\nLast five records:")
display(df.tail())

print("\nShape of dataset:", df.shape)

print("\nData types:")
print(df.dtypes)

print("\nMissing values:")
print(df.isnull().sum())

print("\nSummary statistics:")
display(df.describe())
```

First five records:

Month	Passengers
1949-01-01	112
1949-02-01	118
1949-03-01	132
1949-04-01	129
1949-05-01	121

Last five records:

Month	Passengers
1960-08-01	606
1960-09-01	508
1960-10-01	461
1960-11-01	390
1960-12-01	432

```
Shape of dataset: (144, 1)

Data types:
Passengers    int64
dtype: object

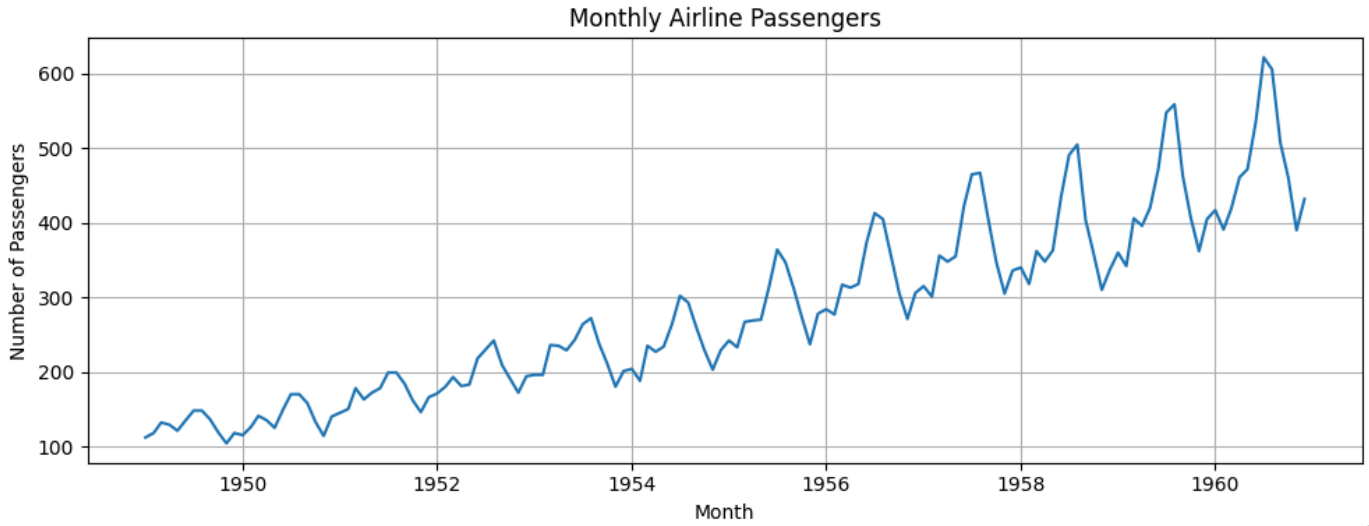
Missing values:
Passengers    0
dtype: int64

Summary statistics:
```

	Passengers
count	144.000000
mean	280.298611
std	119.966317
min	104.000000
25%	180.000000
50%	265.500000
75%	360.500000
max	622.000000

TASK 3: PLOT THE TIME SERIES

```
plt.figure(figsize=(10, 4))
plt.plot(df.index, df["Passengers"])
plt.title("Monthly Airline Passengers")
plt.xlabel("Month")
plt.ylabel("Number of Passengers")
plt.grid(True)
plt.tight_layout()
plt.show()
```




```
TASK 4: CHECK STATIONARITY USING ADF TEST
```

```
def adf_test(series, title):
    result = adfuller(series.dropna())

    print("\n", title)
    print("ADF Statistic:", round(result[0], 4))
    print("p-value:", round(result[1], 6))

    if result[1] <= 0.05:
        print("The series is stationary.")
    else:
        print("The series is non-stationary.")

    return result[1]


p_original = adf_test(
    df["Passengers"],
    "ADF TEST ON ORIGINAL SERIES"
)
```

```
 ADF TEST ON ORIGINAL SERIES
ADF Statistic: 0.8154
p-value: 0.99188
The series is non-stationary.
```

TASK 5: FIRST DIFFERENCING

```
df["First_Difference"] = df["Passengers"].diff()

print("\nFirst-differenced data:")
display(df[["Passengers", "First_Difference"]].head(10))

p_first = adf_test(
    df["First_Difference"],
    "ADF TEST AFTER FIRST DIFFERENCING"
)
```

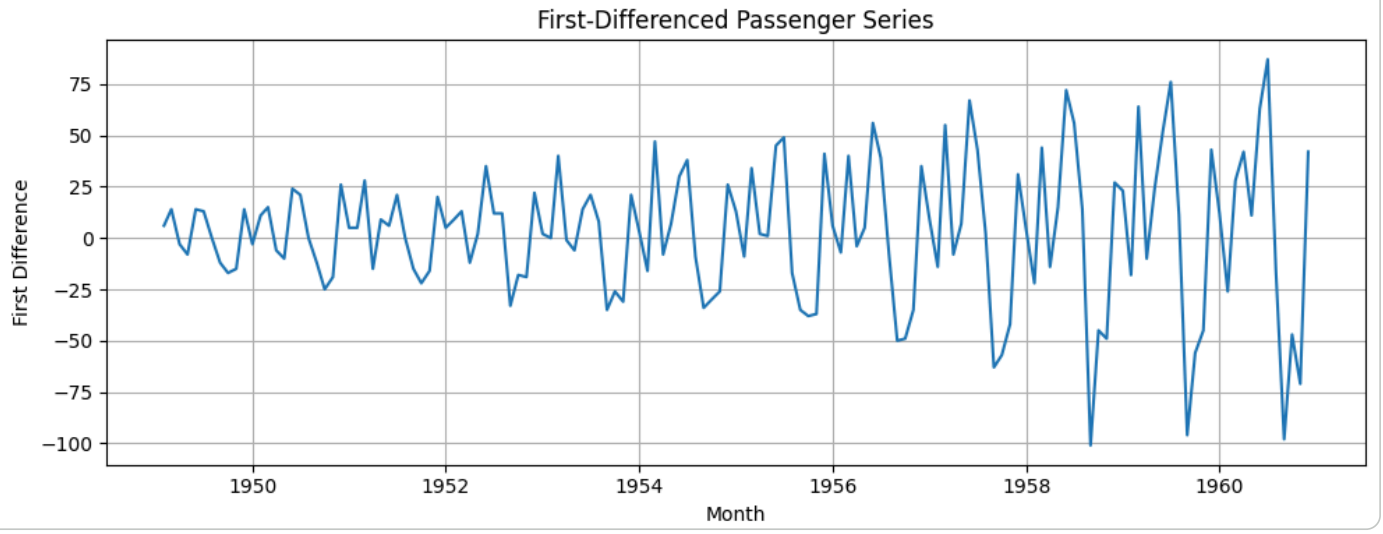
First-differenced data:

Month	Passengers	First_Difference
1949-01-01	112	NaN
1949-02-01	118	6.0
1949-03-01	132	14.0
1949-04-01	129	-3.0
1949-05-01	121	-8.0
1949-06-01	135	14.0
1949-07-01	148	13.0
1949-08-01	148	0.0
1949-09-01	136	-12.0
1949-10-01	119	-17.0

```
 ADF TEST AFTER FIRST DIFFERENCING
ADF Statistic: -2.8293
p-value: 0.054213
The series is non-stationary.
```

TASK 6: VISUALIZE FIRST-DIFFERENCED SERIES

```
plt.figure(figsize=(10, 4))
plt.plot(df.index, df["First_Difference"])
plt.title("First-Differenced Passenger Series")
plt.xlabel("Month")
plt.ylabel("First Difference")
plt.grid(True)
plt.tight_layout()
plt.show()
```



TASK 7: APPLY SECOND DIFFERENCING IF REQUIRED

```
df["Second_Difference"] = df["First_Difference"].diff()

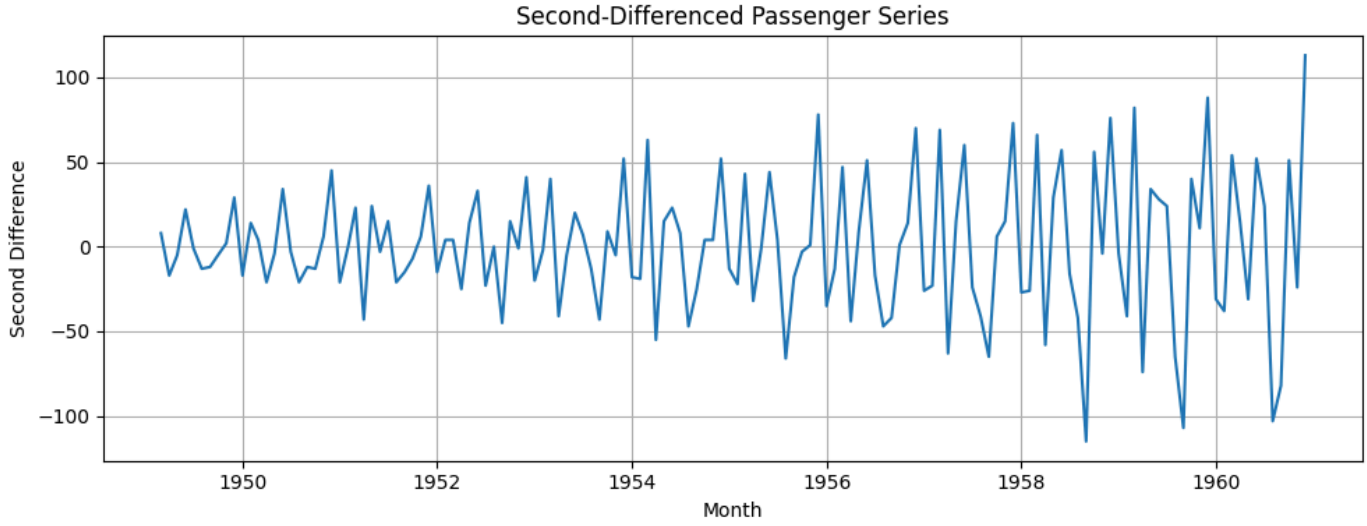
if p_original > 0.05 and p_first > 0.05:
    d = 2
    p_second = adf_test(
        df["Second_Difference"],
        "ADF TEST AFTER SECOND DIFFERENCING"
    )
```

```
    plt.figure(figsize=(10, 4))
    plt.plot(df.index, df["Second_Difference"])
    plt.title("Second-Differenced Passenger Series")
    plt.xlabel("Month")
    plt.ylabel("Second Difference")
    plt.grid(True)
    plt.tight_layout()
    plt.show()

else:
    d = 1 if p_original > 0.05 else 0

print("\nSelected differencing order d =", d)
```

```
 ADF TEST AFTER SECOND DIFFERENCING
ADF Statistic: -16.3842
p-value: 0.0
The series is stationary.
```




```
Selected differencing order d = 2
```

TASK 8: SPLIT ORIGINAL SERIES INTO TRAINING AND TESTING DATA

```
series = df["Passengers"]

split_point = int(len(series) * 0.80)

train = series.iloc[:split_point]
test = series.iloc[split_point:]

print("\nTraining records:", len(train))
print("Testing records:", len(test))

print("\nTraining period:")
print(train.index.min(), "to", train.index.max())

print("\nTesting period:")
print(test.index.min(), "to", test.index.max())
```

```
Training records: 115
Testing records: 29

Training period:
1949-01-01 00:00:00 to 1958-07-01 00:00:00

Testing period:
1958-08-01 00:00:00 to 1960-12-01 00:00:00
```

TASK 9: FIT ARIMA MODEL

```
model = ARIMA(
    train,
    order=(2, d, 2)
)

model_fit = model.fit()

print("\nARIMA Model Summary:")
print(model_fit.summary())
```

```
ARIMA Model Summary:
                               SARIMAX Results
==============================================================================
Dep. Variable:             Passengers   No. Observations:                  115
Model:                 ARIMA(2, 2, 2)   Log Likelihood                -523.084
Date:                Fri, 09 Oct 2026   AIC                           1056.169
Time:                        06:03:42   BIC                           1069.806
Sample:                    01-01-1949   HQIC                          1061.702
                         - 07-01-1958
Covariance Type:                  opg
==============================================================================
                 coef    std err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
ar.L1         -0.5036      0.122     -4.143      0.000      -0.742      -0.265
ar.L2          0.0912      0.099      0.921      0.357      -0.103       0.285
ma.L1         -0.0552     24.247     -0.002      0.998     -47.578      47.468
ma.L2         -0.9447     22.900     -0.041      0.967     -45.828      43.939
sigma2       587.1939   1.43e+04      0.041      0.967   -2.73e+04    2.85e+04
===================================================================================
Ljung-Box (L1) (Q):                   0.00   Jarque-Bera (JB):                 2.25
Prob(Q):                              1.00   Prob(JB):                         0.32
Heteroskedasticity (H):               5.15   Skew:                             0.11
Prob(H) (two-sided):                  0.00   Kurtosis:                         2.34
===================================================================================

Warnings:
[1] Covariance matrix calculated using the outer product of gradients (complex-step).
```

TASK 10: FORECAST TEST DATA

```
forecast_values = model_fit.forecast(steps=len(test))

forecast = pd.Series(
    np.asarray(forecast_values),
    index=test.index,
    name="Predicted Passengers"
)

print("\nActual and Predicted Values:")
results = pd.DataFrame({
    "Actual Passengers": test,
    "Predicted Passengers": forecast
})

display(results)
```

Actual and Predicted Values:

Month	Actual Passengers	Predicted Passengers
1958-08-01	505	508.392539
1958-09-01	404	509.653321
1958-10-01	359	515.516648
1958-11-01	310	517.590598
1958-12-01	337	521.992754
1959-01-01	360	524.876742
1959-02-01	342	528.737670
1959-03-01	406	531.968117
1959-04-01	396	535.605194
1959-05-01	420	538.979977
1959-06-01	472	542.523947
1959-07-01	548	545.958786
1959-08-01	559	549.464018
1959-09-01	463	552.923844
1959-10-01	407	556.412958
1959-11-01	362	559.883181
1959-12-01	405	563.365589
1960-01-01	417	566.840137
1960-02-01	391	570.319755
1960-03-01	419	573.796103
1960-04-01	461	577.274560
1960-05-01	472	580.751657
1960-06-01	535	584.229631
1960-07-01	622	587.707039
1960-08-01	606	591.184813
1960-09-01	508	594.662350
1960-10-01	461	598.140040
1960-11-01	390	601.617632
1960-12-01	432	605.095287

Next steps: Generate code with `results` New interactive sheet

TASK 11: CALCULATE ERROR METRICS

```
mse = mean_squared_error(test, forecast)
mae = mean_absolute_error(test, forecast)
rmse = np.sqrt(mse)

print("\nFORECAST EVALUATION")
print("Mean Squared Error (MSE):", round(mse, 2))
print("Mean Absolute Error (MAE):", round(mae, 2))
print("Root Mean Squared Error (RMSE):", round(rmse, 2))
```

```
FORECAST EVALUATION
Mean Squared Error (MSE): 18381.74
Mean Absolute Error (MAE): 120.27
Root Mean Squared Error (RMSE): 135.58
```

TASK 12: PLOT TRAINING, TESTING AND FORECAST

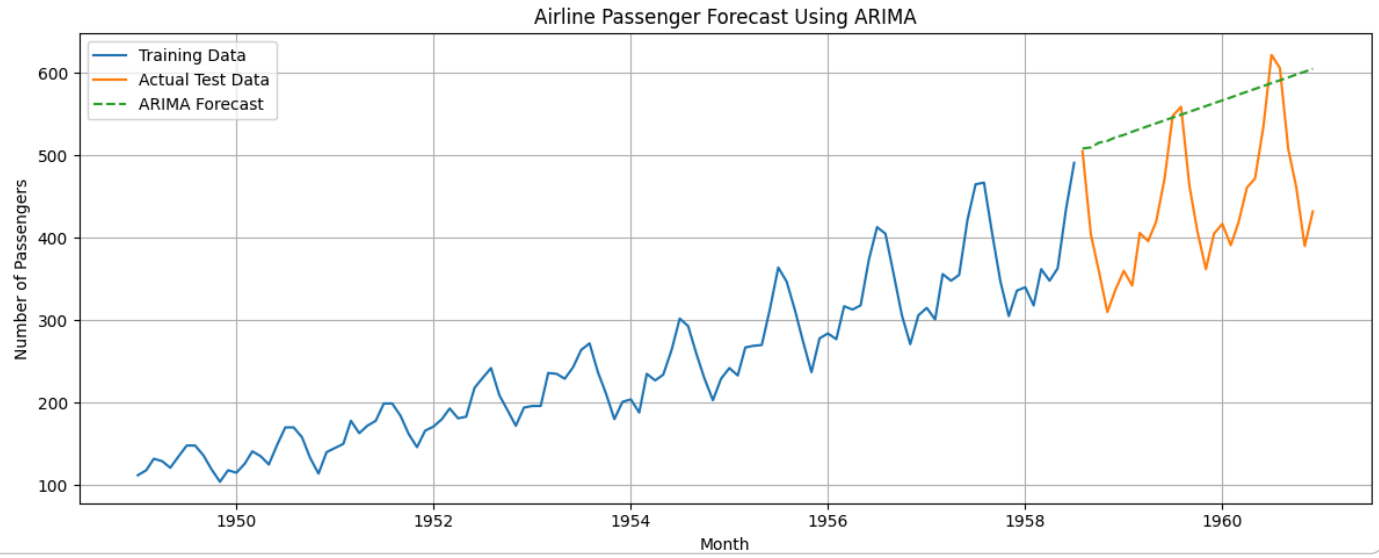
```
plt.figure(figsize=(12, 5))

plt.plot(train.index, train,
         label="Training Data")

plt.plot(test.index, test,
         label="Actual Test Data")

plt.plot(forecast.index, forecast,
         linestyle="--",
         label="ARIMA Forecast")

plt.title("Airline Passenger Forecast Using ARIMA")
plt.xlabel("Month")
plt.ylabel("Number of Passengers")
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.show()
```



TASK 13: DISPLAY FINAL RESULTS

```
print("ARIMA Order:", (2, d, 2))
print("MSE:", round(mse, 2))
print("MAE:", round(mae, 2))
print("RMSE:", round(rmse, 2))
```

```
ARIMA Order: (2, 2, 2)
MSE: 18381.74
MAE: 120.27
RMSE: 135.58
```